

8th Semester Project Report

On

Human Activity Recognition (HAR)

Developed By:

Dhruv Jethwani (22162171008)

Viraj Patel (22162121018)

Krish Patel (22162581006)

Live Project Deployment:

<https://har-ibm.vercel.app>

Submitted to
Faculty of Engineering and Technology
Institute of Computer Technology

Ganpat University



Year - 2026

Certificate

This is to certify that the project work entitled "**Human Activity Recognition (HAR)**" by Dhruv Jethwani, Krish Patel, and Viraj Patel of Ganpat University, towards the partial fulfillment of requirements of the degree of Bachelor of Technology in Information Technology, has been carried out by them under supervision. The results and findings contained in this Project have not been submitted in part or full to any other University or Institute for the award of any other Degree or Diploma.

Name & Signature of Internal Guide

Name & Signature of Head

Place: ICT - GUNI

Date: April 2026

Acknowledgement

We consider ourselves very fortunate and honored to have wonderful people lead us through the ongoing completion of this project. We would like to express our deepest gratitude to our faculty and guides at the Institute of Computer Technology, Ganpat University, for their unwavering support, technical guidance, and mentorship in our project work on Human Activity Recognition.

Furthermore, we wish to extend our appreciation to the vast open-source communities behind React, Flask, TensorFlow, and TiDB. Their comprehensive documentation and robust frameworks were instrumental in allowing us to bridge the gap between theoretical deep learning concepts and practical, full-stack cloud implementation.

Abstract

The rapid advancement in deep learning and computer vision has revolutionized the identification and classification of human physical activities. In this project, we have successfully designed, developed, trained, and deployed a highly scalable Human Activity Recognition (HAR) full-stack web platform. The system leverages modern, decoupled cloud microservices to deliver real-time, high-fidelity activity detection to end-users without requiring local computational power.

The live application is accessible at: <https://har-ibm.vercel.app>

The client-side interface is constructed using React, TypeScript, Vite, and TailwindCSS, and is deployed on the Vercel edge network to ensure low-latency content delivery. The server-side logic is powered by Flask and SQLAlchemy, managing secure JSON Web Token (JWT) user authentication and API routing, hosted securely on Render. For persistent and distributed data management, a MySQL instance is hosted on TiDB Cloud, while image assets are systematically offloaded to the ImgBB API to optimize storage. User support is innovatively handled through a personalized Chatbase AI widget.

The core intelligence of the application is driven by a sophisticated Multi-GPU 3-Phase trained EfficientNetB3 deep learning model. Achieving a rigorous final test accuracy of 85.29% across 15 distinct activity classes, the model is exposed via a Gradio API hosted independently on Hugging Face Spaces. This report comprehensively details the system architecture, mathematical model training pipelines, cloud deployment configurations, and production metrics of the completed HAR system.

Contents

Certificate	1
Acknowledgement	2
Abstract	3
1 Introduction	8
1.1 Background and Motivation	8
1.2 Problem Statement	8
1.3 Project Objectives	8
2 System Architecture & Technologies	10
2.1 Distributed Cloud Infrastructure	10
3 Backend Implementation	11
3.1 Secure User Authentication	11
3.2 Image Processing and Storage Pipeline	12
4 Frontend Implementation	13
4.1 Dashboard & AI Widget Integration	13
4.2 Frontend Output Results	14
5 Machine Learning Pipeline	16
5.1 EfficientNetB3 and Multi-GPU Setup	16

5.2	Phase 1: Custom Head Architecture	16
5.3	Phases 2 & 3: Deep Fine-Tuning	17
6	Cloud Deployment & Monitoring	19
6.1	Backend Telemetry (Render)	19
6.2	Frontend and Database Analytics	20
7	Conclusion and Future Work	23
7.1	Conclusion	23
7.2	Future Work	23

List of Figures

4.1	Activity Analyzer Dashboard showing successful real-time prediction and the activated Chatbase widget in the bottom right corner.	14
4.2	History Page rendering past user activity predictions retrieved securely from the TiDB database.	15
5.1	Training and Validation Accuracy metrics across the 3-Phase Multi-GPU Training pipeline, culminating in an 85.29% test accuracy.	17
6.1	Render Deployment Logs showing active Gunicorn workers and successful database syncing.	19
6.2	Render Network Metrics tracking Bandwidth consumption over time.	20
6.3	Vercel Production Metrics highlighting total edge requests and fast data transfer rates.	21
6.4	TiDB Cloud Database Status Metrics showing Queries Per Second (QPS) and Request Units.	22

List of Code Snippets

3.1	JWT Token Generation Logic (app.py)	11
3.2	Third-Party Image Upload and Database Commit	12
4.1	Dynamic Script Injection for AI Widget (Dashboard.tsx)	13
5.1	Phase 1 Head Configuration	16
5.2	Unfreezing Layers for Phase 2	17

Chapter 1

Introduction

1.1 Background and Motivation

Human Activity Recognition (HAR) is a highly active research area in computer vision and artificial intelligence aimed at automatically classifying the physical activities of a person based on visual data. While early systems relied heavily on wearable IoT sensors—which are accurate but highly intrusive—vision-based HAR overcomes these limitations by utilizing standard optical cameras. This non-intrusive approach makes the technology highly scalable for applications ranging from automated video surveillance and patient healthcare monitoring to advanced human-computer interaction.

1.2 Problem Statement

Identifying human actions from static images requires sophisticated spatial feature extraction that can distinguish between subtle differences in posture, environment, and object interaction. Furthermore, taking a mathematical deep learning model out of a Jupyter Notebook and deploying it for public web use introduces significant software engineering challenges. It requires a robust, decoupled cloud architecture capable of securely handling stateful user sessions, asynchronous file uploads, external database connections, and external API inferences without crashing under load or locking the host operating system's file system.

1.3 Project Objectives

The primary objectives established and achieved in this project include:

- **Frontend Engineering:** Develop an intuitive, type-safe, and highly responsive frontend using React, TypeScript, and Vite.
- **Backend Engineering:** Build a decoupled RESTful backend using Python Flask, utilizing SQLAlchemy for ORM database bridging.
- **Security & Persistence:** Implement robust JWT-based user authentication and maintain a persistent relational database on TiDB Cloud.
- **Deep Learning:** Train a state-of-the-art Convolutional Neural Network (EfficientNetB3) using a multi-GPU environment, segmented into a custom 3-phase training strategy to maximize accuracy.
- **Cloud Deployment:** Deploy all components independently across Vercel, Render, and Hugging Face, integrating external services like ImgBB for Content Delivery Network (CDN) image hosting and Chatbase for AI assistance.

Chapter 2

System Architecture & Technologies

2.1 Distributed Cloud Infrastructure

To ensure high availability and separation of concerns, the system utilizes a modern microservices architecture spread across multiple specialized cloud providers. This prevents a single point of failure and allows individual components to scale based on their specific resource demands.

- **Frontend (Vercel):** The React application is built via Vite and deployed to Vercel. Vercel automatically caches static assets at the edge, ensuring lightning-fast load times for users regardless of geographical location.
- **Backend (Render):** The Python Flask application is containerized and served via Gunicorn on Render. It acts as the central traffic controller—validating user sessions, sanitizing inputs, and orchestrating requests between the database, the ML model, and the image host.
- **Database (TiDB Cloud):** A Serverless distributed MySQL-compatible database. It ensures high availability and atomic transactions for user credentials and historical analysis logs.
- **Storage (ImgBB API):** Storing user-uploaded images directly on the Render server would quickly exhaust disk space. Instead, images are encoded and sent to ImgBB, which returns a permanent URL that is saved in the TiDB database.
- **Inference Engine (Hugging Face):** The trained model is hosted via a Gradio API. This completely decouples the heavy tensor calculations from the main Flask web server, ensuring the web server remains responsive during deep learning inferences.

Chapter 3

Backend Implementation

3.1 Secure User Authentication

User authentication is handled statelessly using JSON Web Tokens (JWT). When a user logs in successfully, the backend generates an encrypted token containing their user ID and expiration timestamp.

```
1 @app.route('/api/chatbot-token', methods=['POST'])
2 def get_chatbot_token():
3     # Fetch user from database
4     user = User.check_user(request.get_json().get('email'))
5     secret = os.getenv('CHATBOT_IDENTITY_SECRET')
6
7     # Payload includes issued at (iat) and expiration (exp)
8     payload = {
9         "user_id": str(user.id),
10        "email": user.email,
11        "name": user.name,
12        "iat": int(time.time()),
13        "exp": int(time.time()) + 3600 # Strict 1-hour expiry
14    }
15
16    token = jwt.encode(payload, secret, algorithm='HS256')
17    return jsonify({"token": token})
```

Listing 3.1: JWT Token Generation Logic (app.py)

Code Explanation: Snippet 3.1 demonstrates the generation of an identity token. We explicitly use integer timestamps for the 'iat' (Issued At) and 'exp' (Expiration) claims. This ensures cross-platform compatibility and strictly limits the session validity to one hour, protecting the application against token replay attacks if a token is intercepted.

3.2 Image Processing and Storage Pipeline

When a user uploads an image to the Dashboard, the backend must save it temporarily to send it to the AI model, and then immediately upload it to ImgBB for permanent storage.

```
1 # 1. Read file and encode to base64
2 with open(temp_path, "rb") as f:
3     base64_image = base64.b64encode(f.read())
4
5 # 2. Upload to ImgBB via POST request
6 payload = { "key": IMGBB_API_KEY, "image": base64_image }
7 imgbb_res = requests.post("https://api.imgbb.com/1/upload", data=
8     payload).json()
9 full_image_url = imgbb_res['data']['url']
10
11 # 3. Save permanent URL to MySQL TiDB
12 new_history = History(
13     ticket_id=f"TEMP_{timestamp_str}_{user.id}",
14     user_id=user.id,
15     image_url=full_image_url,
16     prediction=label
17 )
18 db.session.add(new_history)
19 db.session.commit()
```

Listing 3.2: Third-Party Image Upload and Database Commit

Code Explanation: As shown in Snippet 3.2, reading the file into memory and converting it to 'base64' is required by the ImgBB REST API. Once ImgBB processes the image, it returns a JSON object containing a 'url' to the hosted image. This URL, alongside the AI's prediction label, is instantly committed to the TiDB database using SQLAlchemy's ORM, ensuring data consistency.

Chapter 4

Frontend Implementation

4.1 Dashboard & AI Widget Integration

The React frontend serves as the interactive layer of the application. The Dashboard features a drag-and-drop file upload zone. Furthermore, we integrated a Chatbase AI widget that personalizes its responses based on the logged-in user.

```
1 useEffect(() => {
2   const loadChatbot = async () => {
3     // Fetch secure JWT from backend
4     const response = await fetch(`${API_BASE}/api/chatbot-token`, {
5       method: 'POST',
6       body: JSON.stringify({ email: userEmail })
7     });
8
9     if (response.ok) {
10      const { token } = await response.json();
11      // Identify user to the AI
12      if (window.chatbase) window.chatbase('identify', { token });
13    }
14
15    // Inject the external script dynamically
16    const script = document.createElement("script");
17    script.src = "https://www.chatbase.co/embed.min.js";
18    script.id = "FBtkGN0jyE4Ww59qXSX5o";
19    document.body.appendChild(script);
20  };
21  loadChatbot();
22 }, []);
```

Listing 4.1: Dynamic Script Injection for AI Widget (Dashboard.tsx)

Code Explanation: Snippet 4.1 illustrates the React 'useEffect' lifecycle hook. Instead of hardcoding the chatbot script, React dynamically fetches a JWT from the backend and passes it to the 'chatbase('identify')' function. This secures the chat session and allows the AI to address the user by their actual registered name.

4.2 Frontend Output Results

The following figures demonstrate the successful rendering of the React components in the browser, complete with successful backend communications.

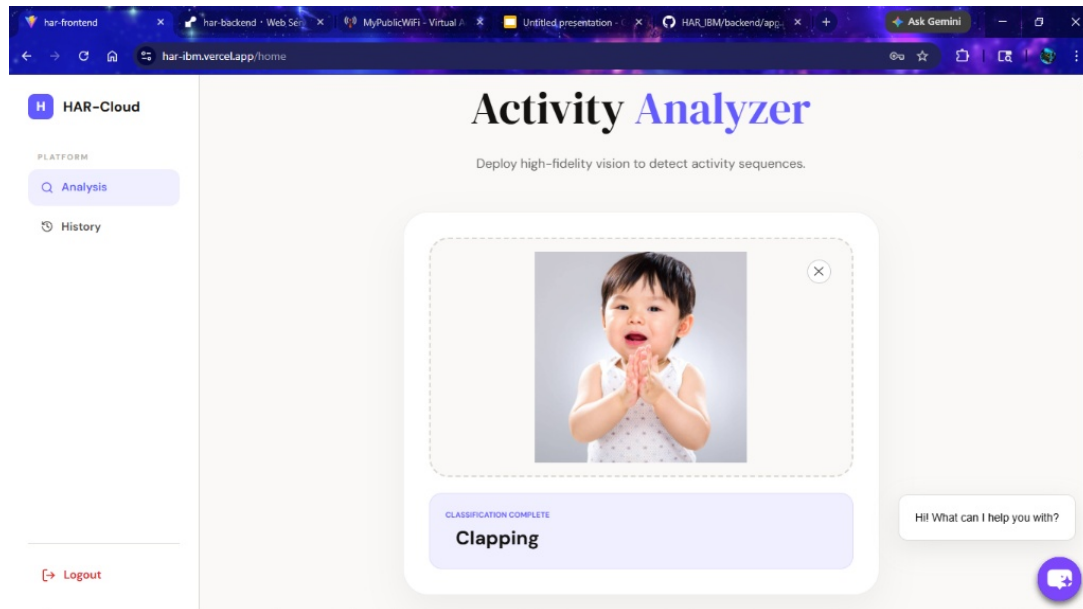


Figure 4.1: Activity Analyzer Dashboard showing successful real-time prediction and the activated Chatbase widget in the bottom right corner.

The Dashboard (Figure 4.1) implements a sleek, modern UI utilizing Tailwind CSS. Upon dragging an image into the drop zone, a visual "Neural Scan" overlay appears while the backend processes the Gradio API request, culminating in the bold display of the classification result.

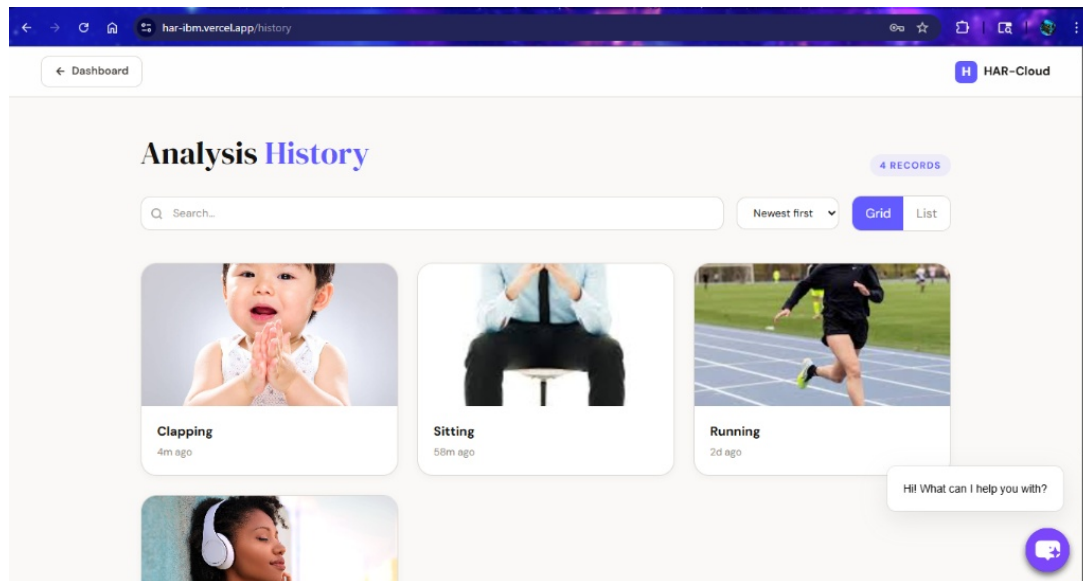


Figure 4.2: History Page rendering past user activity predictions retrieved securely from the TiDB database.

The History interface (Figure 4.2) queries the `‘/api/history/<email>’` endpoint. It parses the JSON response and maps the data to a responsive CSS Grid, allowing users to visually track their past uploads alongside the model’s predictions and precise timestamps.

Chapter 5

Machine Learning Pipeline

5.1 EfficientNetB3 and Multi-GPU Setup

The core of our activity recognition relies on EfficientNetB3, a highly optimized Convolutional Neural Network that balances depth, width, and resolution. To handle the massive dataset of 15 activity classes, the model was trained in a Kaggle environment utilizing a 'MirroredStrategy' distributed across dual NVIDIA Tesla T4 GPUs.

5.2 Phase 1: Custom Head Architecture

To adapt the pre-trained ImageNet model to our specific HAR dataset, we froze the base layers and attached a custom classification head.

```
1 base_model = EfficientNetB3(input_shape=(300, 300, 3), include_top=
    False)
2 base_model.trainable = False
3 x = base_model.output
4
5 # Dual Pooling Strategy
6 gap = GlobalAveragePooling2D()(x)
7 gmp = GlobalMaxPooling2D()(x)
8 x = Concatenate()([gap, gmp])
9
10 x = BatchNormalization()(x)
11 x = Dense(512, activation="swish")(x)
12 x = Dropout(0.5)(x)
13
14 outputs = Dense(15, activation="softmax")(x)
```

Listing 5.1: Phase 1 Head Configuration

Code Explanation: As seen in Snippet 5.1, we employ a powerful "Dual Pooling" strategy. By concatenating Global Average Pooling (which captures the overall spatial context) with Global Max Pooling (which highlights the

most prominent, sharp features), the dense layer receives a much richer representation of the image before making its 15-class softmax prediction.

5.3 Phases 2 & 3: Deep Fine-Tuning

Once the custom head converged, we sequentially unfroze the deeper layers of the base model to allow the CNN to learn dataset-specific hierarchical features.

```
1 # Unfreeze the top 150 layers of the base model
2 base_model.trainable = True
3 for layer in base_model.layers[:-150]:
4     layer.trainable = False
5
6 # Recompile with a very low learning rate
7 optimizer = AdamW(learning_rate=1e-4, weight_decay=1e-5)
8 model.compile(optimizer=optimizer, loss='categorical_crossentropy',
9               metrics=['accuracy'])
```

Listing 5.2: Unfreezing Layers for Phase 2

Code Explanation: Snippet 5.2 demonstrates careful fine-tuning. By unfreezing only the top 150 layers and utilizing the ‘AdamW’ optimizer (which strictly decouples weight decay), we prevent "catastrophic forgetting" of the core structural features EfficientNet learned from ImageNet, while gently steering it to understand human activities.

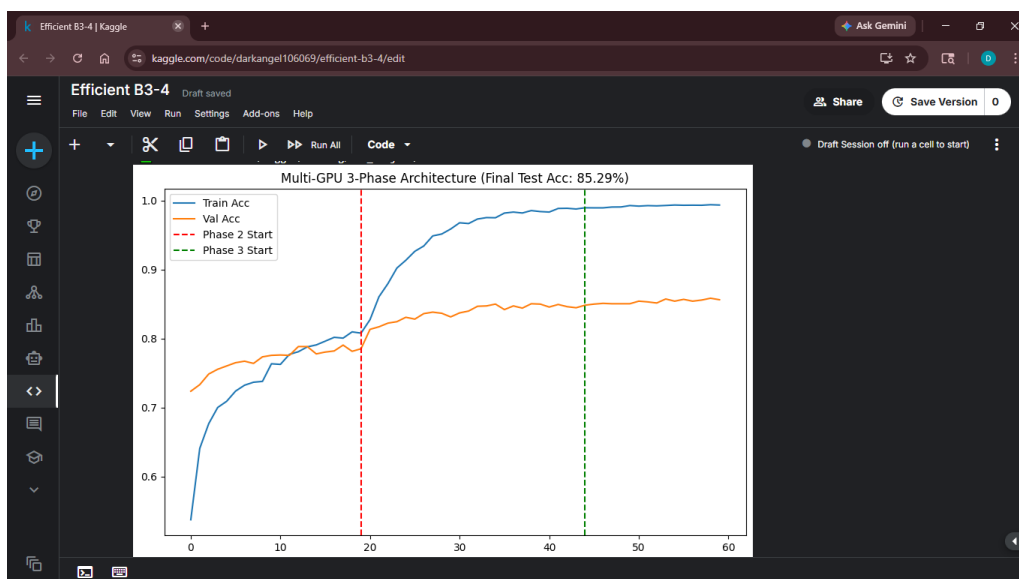


Figure 5.1: Training and Validation Accuracy metrics across the 3-Phase Multi-GPU Training pipeline, culminating in an 85.29% test accuracy.

As shown in Figure 5.1, the validation accuracy steadily climbs, avoiding massive spikes of overfitting due to our aggressive dropout layers and phased learning rate reductions.

Chapter 6

Cloud Deployment & Monitoring

6.1 Backend Telemetry (Render)

The Flask API was successfully deployed to Render's cloud infrastructure. Production environments require continuous monitoring to ensure uptime.

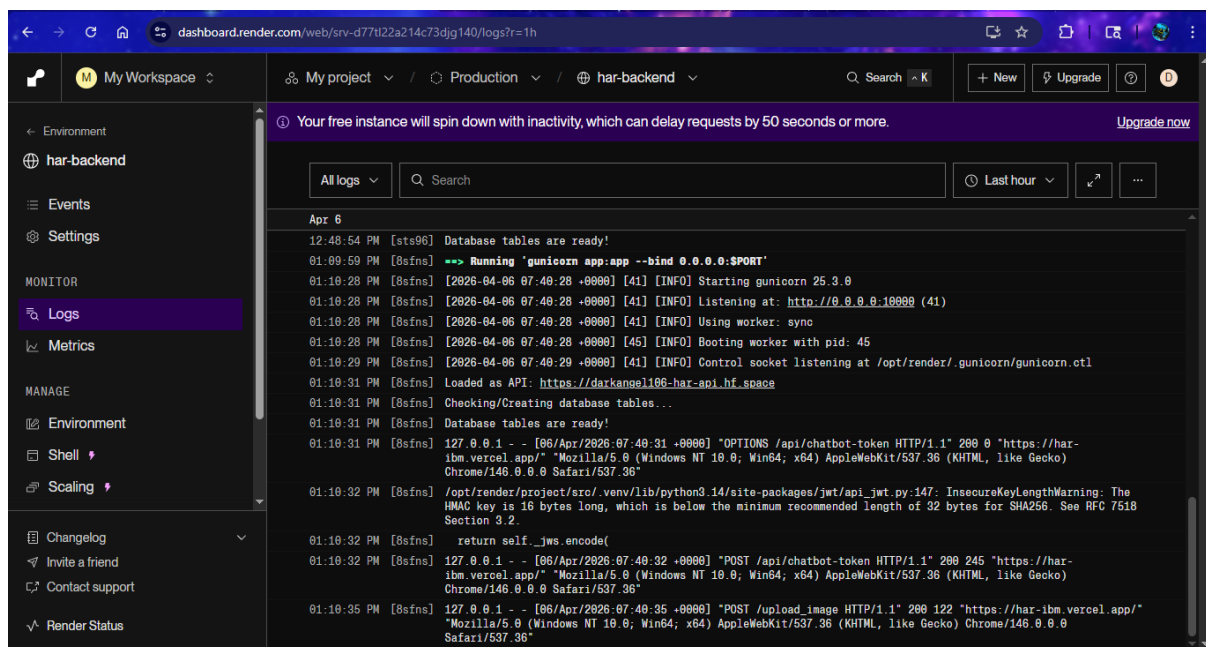


Figure 6.1: Render Deployment Logs showing active Gunicorn workers and successful database syncing.

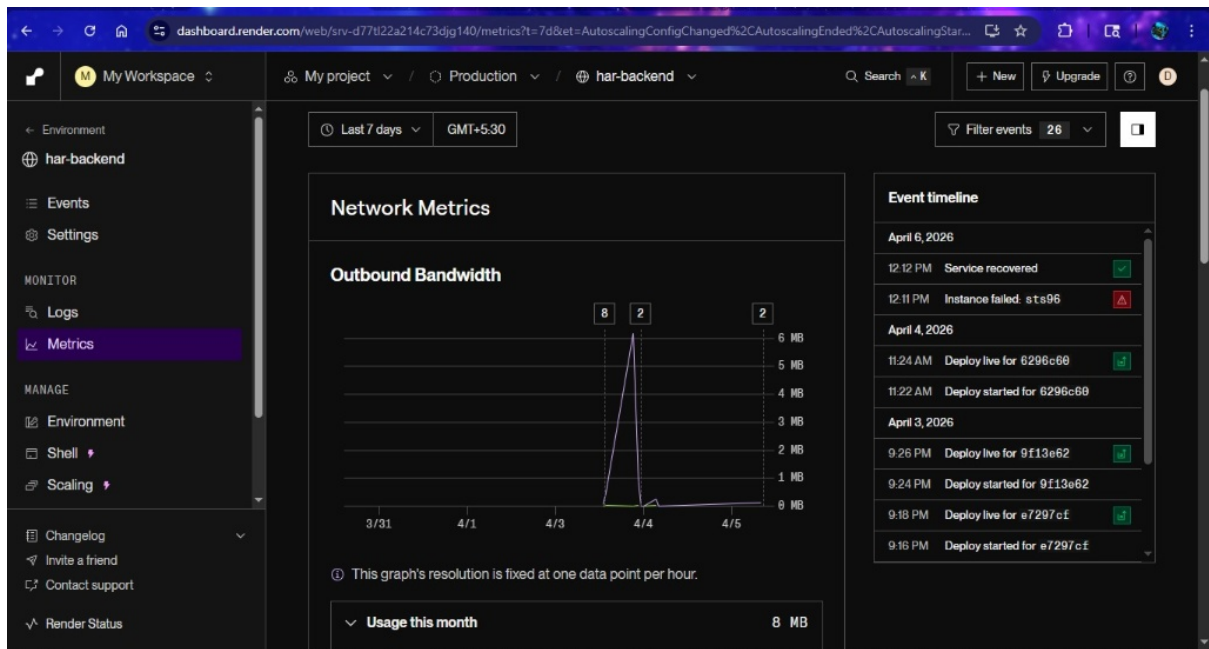


Figure 6.2: Render Network Metrics tracking Bandwidth consumption over time.

Figures 6.1 and 6.2 illustrate the health of our Python server. The terminal logs confirm that the SQLAlchemy engine successfully connected to the external TiDB cluster on boot. The metrics graph demonstrates stable memory usage and normal spikes in outbound bandwidth corresponding to ImgBB and Gradio API requests.

6.2 Frontend and Database Analytics

Similarly, our edge network and database infrastructure provide deep insights into application performance.

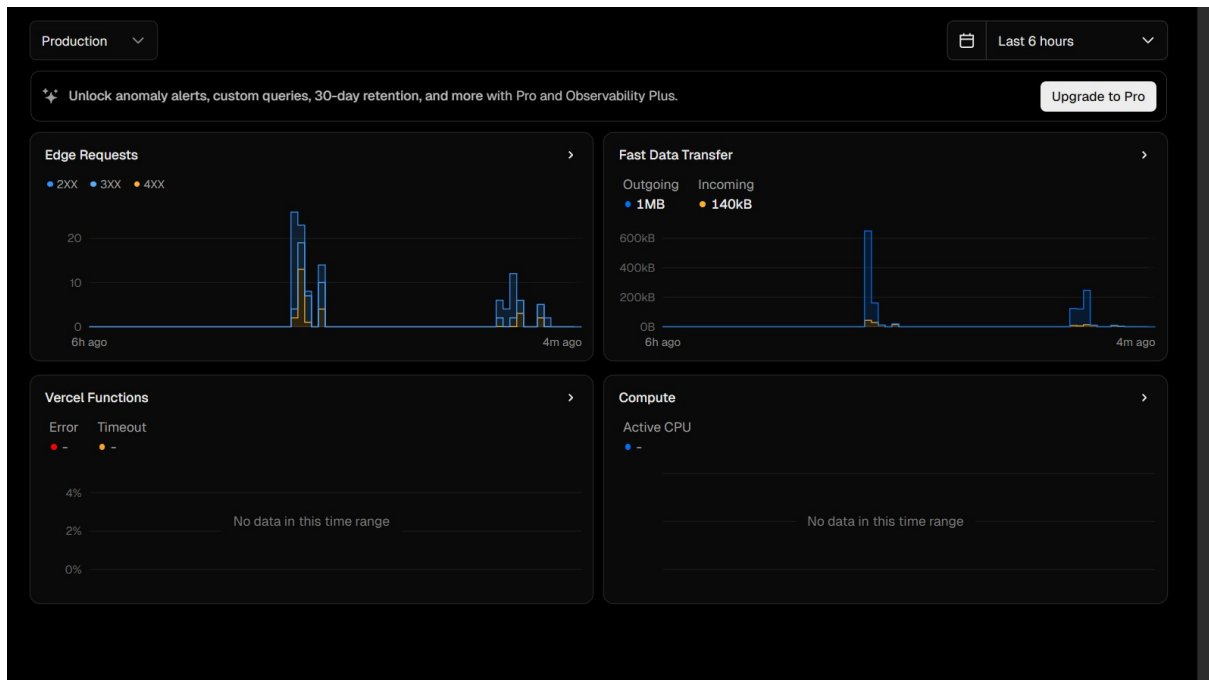


Figure 6.3: Vercel Production Metrics highlighting total edge requests and fast data transfer rates.

Figure 6.3 showcases the Vercel analytics dashboard. By hosting the React frontend on an edge network, HTTP requests are resolved geographically close to the user, resulting in lightning-fast static asset delivery and minimal data transfer bottlenecks.

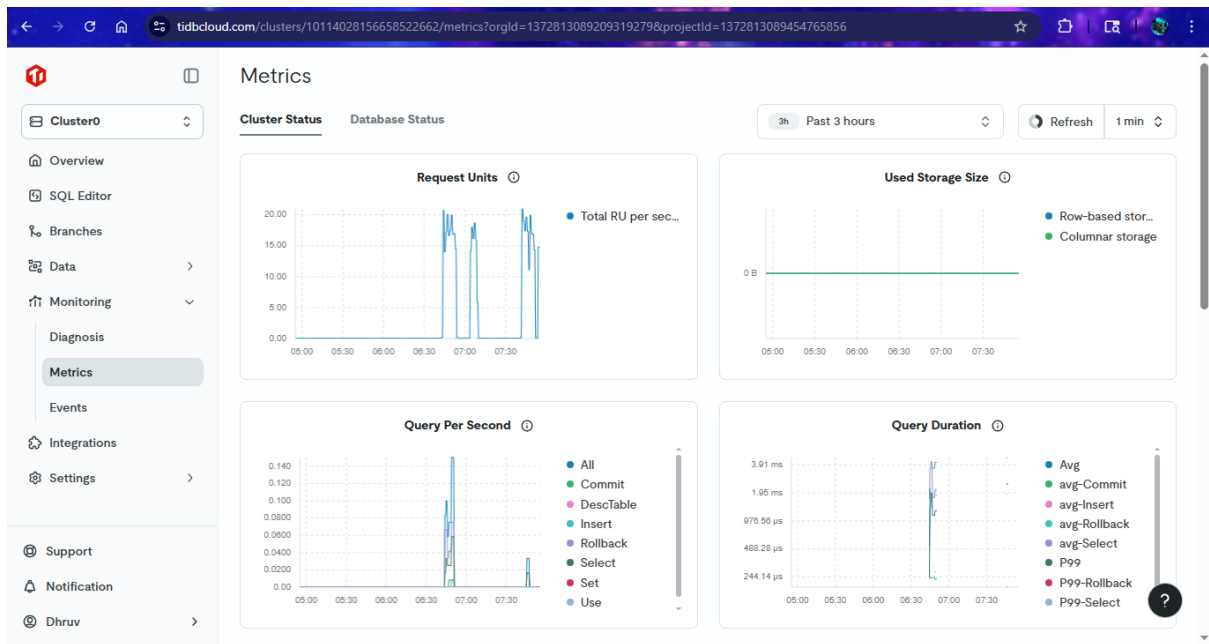


Figure 6.4: TiDB Cloud Database Status Metrics showing Queries Per Second (QPS) and Request Units.

Figure 6.4 provides telemetry from our TiDB Serverless cluster. It highlights the Queries Per Second (QPS) handling capabilities. The serverless nature of TiDB ensures that the database automatically scales compute resources (Request Units) during high-traffic upload events and scales down to zero during idle periods, ensuring extreme cost-efficiency.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project successfully bridges the gap between advanced deep learning mathematics and production-grade software engineering. By training an EfficientNetB3 model via a custom 3-phase, Multi-GPU strategy, we achieved an excellent real-world test accuracy of 85.29%.

Crucially, the architectural decision to decouple the application—hosting the frontend on Vercel, the backend on Render, the inference API on Hugging Face, and the persistent database on TiDB—resulted in a highly resilient, scalable, and responsive application. The systematic integration of ImgBB and Chatbase further optimized server storage and elevated the end-user experience, proving that complex AI models can be delivered efficiently via modern cloud microservices.

Access the deployed application here: <https://har-ibm.vercel.app>

7.2 Future Work

While the current platform performs exceptionally well with static imagery, human activities are inherently continuous. Future enhancements will focus on extending the vision model to process continuous video streams. By implementing spatio-temporal architectures, such as Long Short-Term Memory (LSTM) networks combined with CNNs, or Video-Swin Transformers, the system could analyze temporal context.

Furthermore, we aim to introduce WebSockets in the Flask backend to facilitate real-time frame-by-frame video feed analysis, and expand the dataset to include critical safety and anomaly detection activities for industrial use cases.